



Universidad Tecmilenio
Actividad 3
Estructura de datos
Canek Lugo Soto
2967116

Actividad 3

Primer problema: Serie de Fibonacci recursiva

```
1 package Actividad_3;
2
3 public class problema1 {
4     public static int fibonacci(int n) {
5         if (n <= 1) {
6             return n; // Caso base
7         }
8         return fibonacci(n - 1) + fibonacci(n - 2); // Caso recursivo
9     }
10
11     Run main | Debug main | Run | Debug
12     public static void main(String[] args) {
13         int n = 10;
14         System.out.println("Fibonacci de " + n + " es: " + fibonacci(n));
15     }
16 }
```

En este programa se define una función recursiva que calcula el enésimo número en la serie de Fibonacci. El caso base devuelve el propio número cuando es 0 o 1. El caso recursivo llama nuevamente a la función con los dos valores anteriores y los suma. La recursión sigue hasta llegar a los casos base.

Segundo problema: Suma de subconjuntos

```
1 package Actividad_3;
2
3 public class problema2 {
4     public static boolean subsetSum(int[] arr, int n, int target) {
5         if (target == 0) return true;
6         if (n == 0 && target != 0) return false;
7         // Excluir o incluir el último elemento
8         return subsetSum(arr, n - 1, target) || subsetSum(arr, n - 1, target - arr[n - 1]);
9     }
10
11     Run | Debug
12     public static void main(String[] args) {
13         Run main | Debug main
14         int[] arr = {3, 34, 4, 12, 5, 2};
15         int target = 9;
16         System.out.println("¿Existe subconjunto con suma " + target + "? " + subsetSum(arr, arr.length, target));
17     }
18 }
19
20 }
```

Para este código el algoritmo utiliza recursión para explorar subconjuntos de un arreglo. Si el objetivo es cero, devuelve verdadero, y si no quedan elementos retorna falso. En cada paso, se decide incluir o excluir el último número, verificando si alguna de esas opciones alcanza el valor objetivo. Usa backtracking implícito mediante recursión.

Tercer problema: Algoritmo de backtracking

```
1  package Actividad_3;
2
3  public class problema3 {
4      private static final int SIZE = 9;
5
6      public static boolean solveSudoku(int[][] board, int row, int col) {
7          if (row == SIZE - 1 && col == SIZE) return true;
8          if (col == SIZE) {
9              row++;
10             col = 0;
11         }
12         if (board[row][col] != 0) return solveSudoku(board, row, col + 1);
13
14         for (int num = 1; num <= SIZE; num++) {
15             if (isSafe(board, row, col, num)) {
16                 board[row][col] = num;
17                 if (solveSudoku(board, row, col + 1)) return true;
18                 board[row][col] = 0;
19             }
20         }
21         return false;
22     }
23
24     private static boolean isSafe(int[][] board, int row, int col, int num) {
25         for (int x = 0; x < SIZE; x++) {
26             if (board[row][x] == num || board[x][col] == num) return false;
27         }
28         int startRow = row - row % 3, startCol = col - col % 3;
29         for (int i = 0; i < 3; i++) {
30             for (int j = 0; j < 3; j++) {
31                 if (board[i + startRow][j + startCol] == num) return false;
32             }
33         }
34         return true;
35     }
36
37     public static void printBoard(int[][] board) {
38         for (int r = 0; r < SIZE; r++) {
39             for (int d = 0; d < SIZE; d++) {
40                 System.out.print(board[r][d] + " ");
41             }
42             System.out.println();
43         }
44     }
45 }
```

```

Run main | Debug main | Run | Debug
46  ▾ public static void main(String[] args) {
47  ▾     int[][] board = {
48      {5,3,0,0,7,0,0,0,0},
49      {6,0,0,1,9,5,0,0,0},
50      {0,9,8,0,0,0,0,6,0},
51      {8,0,0,0,6,0,0,0,3},
52      {4,0,0,8,0,3,0,0,1},
53      {7,0,0,0,2,0,0,0,6},
54      {0,6,0,0,0,0,2,8,0},
55      {0,0,0,4,1,9,0,0,5},
56      {0,0,0,0,8,0,0,7,9}
57  };
58
59      if (solveSudoku(board, row:0, col:0))
60      |   printBoard(board);
61      else
62      |   System.out.println(x:"No existe solución.");
63  }
64  }
65

```

Para este programa resuelve un Sudoku mediante backtracking. Verifica celda por celda, intentando colocar valores válidos. La función `isSafe` comprueba filas, columnas y subcuadros 3x3. Si un valor es seguro, se asigna y continúa el proceso recursivo. Si falla, retrocede y prueba otro número hasta completar satisfactoriamente el tablero.